🏦 Modelo de Aprovação de Empréstimos

Um guia completo sobre como a Inteligência Artificial ajuda bancos a tomar decisões melhores

📑 Índice

1. O Problema de Negócio
2. Conhecendo os Dados
3. Análise Exploratória
4. Pré-processamento
5. Modelo Baseline (Perceptron)
6. Matriz de Confusão
7. Árvore de Decisão
8. Regularização e Overfitting
9. Validação Cruzada
10. Random Forest
11. Curva ROC e AUC
12. Curva Precision-Recall
13. Impacto Financeiro
14. Conclusão

1. 🎯 O Problema de Negócio

Você já pediu um empréstimo no banco? Lembra de preencher formulários, apresentar comprovantes de renda, e esperar a resposta? Por trás dessa decisão, existe um processo complexo.

Agora imagine o lado do banco: **milhares de pessoas pedem empréstimos todo dia**. Se aprovar todo mundo, o banco quebra com calotes. Se rejeitar todo mundo, não ganha dinheiro. A pergunta de milhões é: **quem vai pagar e quem vai dar calote?**

45.000	22%	78%
Solicitações analisadas	São aprovadas	São rejeitadas

Distribuição das Decisões

78% Rejeitados (35.000 clientes)

22% Aprovados (10.000 clientes)

Os dois tipos de erro que o banco pode cometer

Pense assim: quando você vai ao médico fazer um exame, ele pode errar de duas formas - dizer que você está doente quando não está, ou dizer que está saudável quando na verdade está doente. No banco é parecido:

❌ **Erro 1: Aprovar quem vai dar calote (Falso Positivo)**

O banco empresta R$ 10.000 para alguém que parecia confiável, mas nunca paga. **Prejuízo direto - o dinheiro foi embora.**

⚠️ **Erro 2: Rejeitar quem pagaria direitinho (Falso Negativo)**

O banco rejeita um cliente que pagaria todas as parcelas em dia. **Oportunidade perdida - deixou de ganhar os juros.**

⚖️ **O equilíbrio é crucial:** Ser cauteloso demais = perder clientes bons e lucro. Ser permissivo demais = perder dinheiro com calotes. O objetivo é encontrar o ponto ideal.

Por que não deixar um gerente humano decidir tudo?

Gerentes são ótimos, mas têm limitações:

🐢

Escala

Um gerente analisa 50 casos por dia. O banco recebe 1.000.

😴

Consistência

Na segunda de manhã decide diferente da sexta à tarde.

🧠

Objetividade

Pode ter preconceitos sem perceber (idade, bairro, aparência).

🤖 **O que a Inteligência Artificial faz:** Olha para **36.000 empréstimos do passado** onde já sabemos quem pagou e quem deu calote. Encontra padrões: "pessoas com essas características costumam pagar" ou "pessoas com aquelas características costumam dar calote". Depois, aplica esses padrões para novos clientes - de forma rápida

> (segundos), consistente (mesma regra para todos) e objetiva (só olha os números, não a cara da pessoa).

2. 📋 Conhecendo os Dados

Lembra quando você pediu empréstimo e preencheu aquele formulário enorme? O banco coleta essas informações por um motivo: **são as "pistas" que ajudam a prever se você vai pagar ou não**.
No nosso estudo, temos **13 informações** sobre cada cliente:

Variável	O que significa	Tipo
person_age	Idade do cliente	Número
person_income	Renda mensal	Número
person_emp_exp	Anos de experiência profissional	Número
loan_amnt	Valor do empréstimo solicitado	Número
loan_int_rate	Taxa de juros oferecida	Número
loan_percent_income	% da renda comprometida	Número
credit_score	Score de crédito (300-850)	Número
person_gender	Gênero	Categoria
person_education	Escolaridade	Categoria
person_home_ownership	Situação de moradia	Categoria

loan_intent	Finalidade do empréstimo	Categoria
previous_loan_defaults	Já deu calote antes?	Categoria

🎯 **Variável-alvo:** `loan_status` = 0 (rejeitado) ou 1 (aprovado)

3. 🔍 Análise Exploratória: Conhecendo os Clientes

Antes de ensinar qualquer coisa para a IA, precisamos **entender quem são nossos clientes**. É como um detetive examinando as evidências antes de resolver o caso.

Encontrando dados estranhos (Outliers)

Às vezes os dados têm erros ou valores muito fora do comum. Por exemplo: encontramos um cliente com **144 anos de idade**. Isso claramente é um erro de digitação! Se não tratarmos esses casos, a IA pode aprender coisas erradas.

Idade	Renda	Valor Empréstimo	Credit Score
(outlier: 144 anos)	(outlier: R$7M)	(poucos outliers)	(distribuição OK)

⚠️ **Por que isso importa?**

Imagine calcular a média de idade dos clientes. Com alguém de 144 anos no meio, a média fica distorcida! Por isso usamos técnicas que ignoram esses valores extremos na hora de padronizar os dados.

Descobrindo conexões entre as informações

Algumas informações estão conectadas. Por exemplo: **quanto mais velha a pessoa, mais experiência de trabalho ela tem**. Isso é óbvio, certo? A IA também precisa entender essas conexões.

	age	income	exp	loan_amt	int_rate	%_income	score	TARG
age	1.00	0.07	0.95	0.02	-0.01	-0.04	0.54	-0.02
income	0.07	1.00	0.05	0.24	-0.02	-0.17	0.07	-0.06
%_income	-0.04	-0.17	-0.03	0.39	0.35	1.00	0.02	0.38
int_rate	-0.01	-0.02	-0.01	0.07	1.00	0.35	-0.28	0.33

💡

O que a correlação nos diz?

- **Idade e Experiência = 0.95:** Quase a mesma coisa (pessoas mais velhas têm mais experiência)
- **% Renda com Target = 0.38:** Comprometimento de renda influencia aprovação
- **Taxa de Juros com Target = 0.33:** Taxas maiores correlacionam com aprovação
- **Nenhuma variável domina sozinha:** Máximo é 0.38 - precisamos combinar várias

4. ⚙️ Preparando os Dados para a IA

Antes de ensinar a IA, precisamos preparar os dados. É como organizar o material antes de estudar para uma prova.

Separando dados para "estudar" e para "provar"

Não podemos avaliar um aluno usando as mesmas questões que ele estudou, certo? Com a IA é igual: separamos os dados em dois grupos.

Treino (80%)

36.000 clientes

A IA estuda esses casos para aprender os padrões

+

Teste (20%)

9.000 clientes

Usamos para "provar" se a IA realmente aprendeu

> 💡 **Por que separar?** Se a IA vir todos os dados durante o aprendizado, ela pode "decorar" as respostas em vez de aprender o padrão. Testando com 9 mil clientes que ela nunca viu, garantimos que ela realmente aprendeu.

Transformando texto em números

A IA só entende números. Quando temos informações como "mora de aluguel" ou "mora em casa própria", precisamos converter isso para números. Fazemos assim:

> **Antes:** home_ownership = "RENT"
>
> **Depois:**
> • home_ownership_OWN = 0
> • home_ownership_RENT = 1
> • home_ownership_MORTGAGE = 0

Variável Original	Categorias	Colunas Criadas
person_gender	2	1
person_education	5	4
person_home_ownership	4	3

loan_intent	6	5
previous_loan_defaults	2	1
TOTAL		**14 novas + 8 numéricas = 22 features**

5. 🔪 Primeiro Modelo: Perceptron (O Mais Simples)

Por que começar pelo mais simples? Porque ele serve como referência. Se um modelo complexo não for muito melhor que o simples, não vale a pena a complexidade.
O Perceptron funciona assim: **soma todas as informações do cliente com pesos diferentes**. Se a soma for positiva, aprova. Se for negativa, rejeita.

```
decisão = bias + (peso₁ × idade) + (peso₂ × renda) + ...

Se decisão ≥ 0 → APROVAR
Se decisão < 0 → REJEITAR
```

Coeficientes Aprendidos (Pesos)

previous_defaults	-8.30 🚨
home_own	-0.90
loan_intent_venture	-0.70
loan_%_income	+0.30
loan_int_rate	+0.27

🚨 **Histórico de calote domina!** Peso -8.30 é 10x maior que qualquer outro. Se o cliente já deu calote, praticamente garante rejeição.

Resultado

66%	-0.10
F1-Score	Bias (tendência a rejeitar)

6. 🎯 Entendendo os Acertos e Erros

A "Matriz de Confusão" é uma tabela que mostra **exatamente onde o modelo acertou e onde errou**. Com ela, sabemos quantos caloteiros foram identificados, quantos bons clientes foram aprovados, e onde o modelo se confundiu.

	Previu: Rejeitar	Previu: Aprovar
Real: Rejeitado	**VN = 6.739** ✅ Rejeitou caloteiro	**FP = 261** ❌ Aprovou caloteiro
Real: Aprovado	**FN = 876** ⚠️ Rejeitou bom cliente	**VP = 1.124** ✅ Aprovou bom cliente

Métricas Derivadas

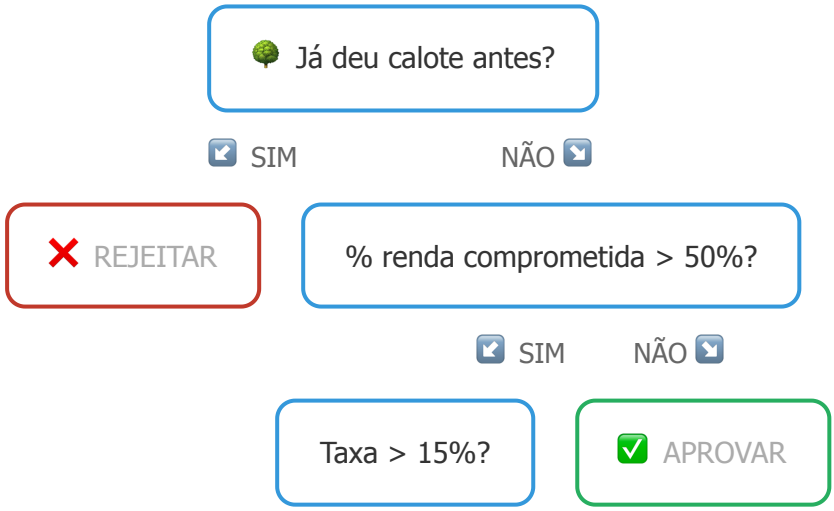
```
Precision = VP / (VP + FP) = 1124 / 1385 = 81%
"Dos que aprovei, quantos realmente pagaram?"
```

```
Recall = VP / (VP + FN) = 1124 / 2000 = 56%
"Dos bons clientes, quantos eu aprovei?"

F1-Score = média harmônica = 66%
```

7. 🌳 Segundo Modelo: Árvore de Decisão

Por que testar outro modelo? O Perceptron é limitado - ele só consegue traçar uma "linha reta" para separar os clientes. A Árvore de Decisão é mais flexível: ela faz **perguntas em sequência**, como um fluxograma.
Vantagem: É fácil de explicar. Você consegue mostrar para o cliente exatamente por que foi rejeitado.



Resultado

100%	77%	23%
F1 no Treino	F1 no Teste	GAP (Overfitting!)

8. 🛡️ O Problema: A Árvore "Decorou" os Dados

A Árvore sem limites acertou **100% no treino**, mas apenas **77% no teste**. O que aconteceu? **Ela decorou** os casos que viu, em vez de aprender o padrão. Criou regras muito específicas como "se idade = 34 e renda = 5.432 e..." que funcionam perfeitamente nos dados de treino mas falham com clientes novos.

A solução: Limitar a complexidade

😈 Sem Limites

Decorou os dados
100% no treino, 77% no teste
Regras específicas demais

→

😇 Com Limites

Aprendeu o padrão
79% no treino, 78% no teste
Regras gerais que funcionam

Os "Controles" da Regularização

Parâmetro	O que faz	Valor usado
max_depth	Limita profundidade da árvore	5 níveis
min_samples_split	Mínimo de casos para criar regra	50
min_samples_leaf	Mínimo de casos em cada folha	20

Comparação: Antes vs Depois

Sem Regularização:

Treino	100%
Teste	77%

Com Regularização:

Treino	79%
Teste	78%

> ✅ Gap reduziu de 23% para 1%. O modelo agora **generaliza bem**!

9. 🔄 Testando Várias Vezes para Ter Certeza

Por que não basta testar uma vez? Pode ter sido sorte! O conjunto de teste pode ter sido "fácil" por acaso.
A solução é **dividir os dados de 5 formas diferentes** e testar todas. Se o modelo for bom em todas, temos certeza de que funciona.

Rodada 1:

TESTE TREINO

Rodada 2:

TREINO TESTE TREINO

... e assim por diante (5 rodadas)

Resultado: Intervalo de Confiança

82.5%	±0.8%	[82-83%]
F1 Médio	Desvio Padrão	IC 95%

📊 **Intervalo de Confiança estreito** = modelo estável e previsível. Podemos garantir F1 entre 82% e 83% em produção.

10. 🌲🌲🌲 Terceiro Modelo: Random Forest (Floresta de Árvores)

A ideia: Uma árvore sozinha pode errar. Mas e se criarmos **200 árvores diferentes** e cada uma der sua opinião? A decisão final é por **votação** - a maioria vence.
Por que funciona melhor? Cada árvore vê os dados de um jeito diferente. Se uma erra, as outras compensam. É o mesmo princípio de pedir opinião para várias pessoas antes de tomar uma decisão importante.

1 **200 amostras aleatórias (Bootstrap)**

Cada árvore treina com dados diferentes

2 **Variáveis aleatórias**

Cada árvore usa um subconjunto das 22 variáveis

3 **Votação**

As 200 árvores votam. A maioria vence!

Feature Importance (O que o modelo aprendeu)

previous_defaults	33,7%
loan_%_income	19,7%
loan_int_rate	16,1%
person_income	11,3%
home_ownership	7,3%

🎯 **Top 5 = 88% da importância!** Gênero e escolaridade têm menos de 1% cada.

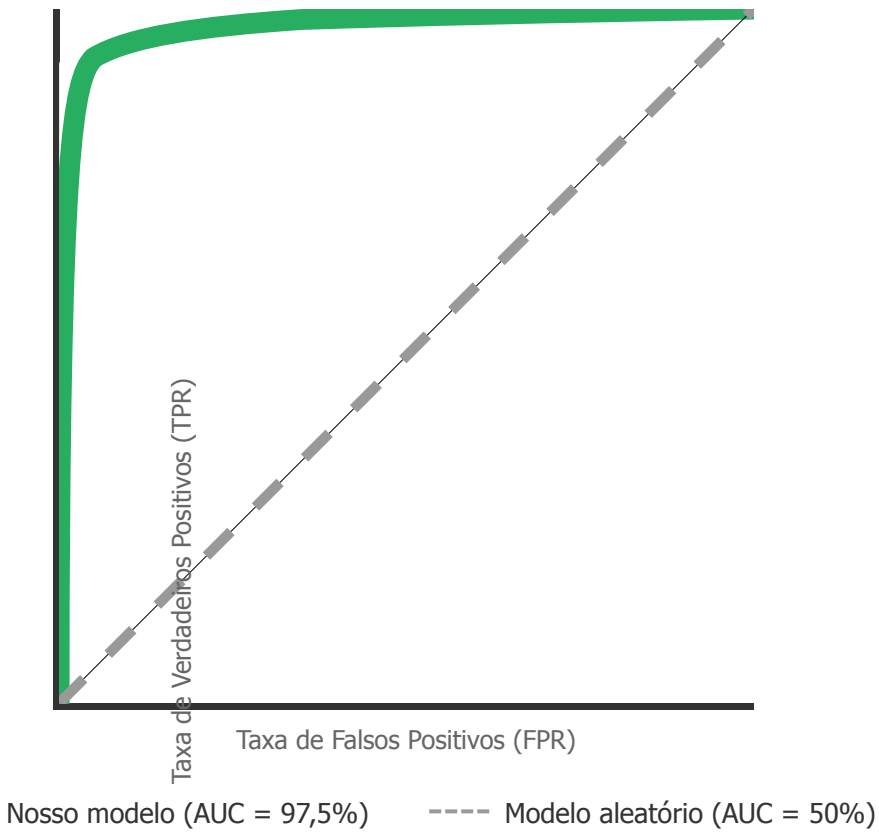
11. 📈 Medindo a Qualidade Geral do Modelo

O modelo não diz apenas "aprovado" ou "rejeitado". Ele dá uma **probabilidade**: "73% de chance de pagar". Cabe ao banco decidir o ponto de corte.

Exemplo: Se o banco quiser ser conservador, só aprova quem tem mais de 80% de chance. Se quiser crescer, aprova a partir de 50%.

Threshold	Estratégia	Efeito
70%	Conservadora	Aprova menos, mais certeza
50%	Equilibrada	Padrão
30%	Agressiva	Aprova mais, aceita risco

Curva ROC: Performance em TODOS os thresholds



50%	70-80%	80-90%
Modelo aleatório	Aceitável	Bom

97,5%
Nosso modelo! 🎉

💡

O que 97,5% de AUC significa?

Se pegarmos aleatoriamente 1 bom pagador e 1 caloteiro, o modelo acerta quem é quem em **97,5% das vezes**!

12. 🎯 O Dilema: Aprovar Mais ou Errar Menos?

Existe um trade-off que o banco precisa aceitar:

Se o banco quiser...	O que acontece...
Aprovar o máximo de bons clientes	Vai acabar aprovando alguns caloteiros também
Evitar ao máximo aprovar caloteiros	Vai acabar rejeitando alguns bons clientes

Não dá para ter os dois ao máximo. O banco precisa escolher sua estratégia baseado no que é mais importante para o negócio.

> ⚖️ **F1-Score** é uma métrica que equilibra os dois lados. Só é alto quando o modelo acerta bem nos dois sentidos.

13. 💰 Impacto Financeiro: O Que Realmente Importa

No final, o banco quer saber: **quanto dinheiro cada modelo me faz perder?**
Calculamos dois tipos de perda:

Tipo de Erro	O que acontece na prática	Quanto perde
Aprovou caloteiro	Emprestou R$10.000 e não recebeu de volta	**R$10.000** (valor total)
Rejeitou bom cliente	Deixou de emprestar e ganhar os juros	**R$1.200** (só os juros de ~12%)

💡 **Perceba:** Aprovar um caloteiro custa ~8x mais do que rejeitar um bom cliente. Por isso o banco tende a ser conservador - é melhor perder oportunidades do que perder dinheiro.

Comparação Financeira dos Modelos

Prejuízo (FP) - Empréstimos dados a maus pagadores

Árvore s/ reg	R$4.53M
Perceptron	R$3.60M
RF Otimizado	**R$1.63M**
Random Forest	**R$1.46M ✓**

Receita Perdida (FN) - Bons pagadores rejeitados

Perceptron	R$0.83M
Random Forest	**R$0.54M**
Árvore s/ reg	**R$0.49M**
RF Otimizado	**R$0.47M ✓**

Impacto Total

Árvore s/ reg	R$5.02M
Perceptron	R$4.43M
RF Otimizado	**R$2.10M**
Random Forest	**R$2.00M ✓**

O Trade-off Descoberto

RF Otimizado

F1: **82,5%** ✓
Menor FN: R$0,47M ✓
Maior FP: R$1,63M
Total: R$2,10M

VS

Random Forest

F1: 80,5%
Menor FP: R$1,46M ✓
Maior FN: R$0,54M
Total: R$2,00M ✓

🤔 **O modelo mais preciso (RF Otimizado) não é o com menor impacto!**

RF Otimizado aprova mais → menos receita perdida, mas mais prejuízo com caloteiros.

14. 🏁 Conclusão: Qual Modelo Escolher?

Testamos 4 modelos diferentes, cada um mais sofisticado que o anterior. Veja como a precisão foi aumentando:

Evolução dos Modelos (F1-Score no Teste)

Perceptron	66%
Árvore	77%
Random Forest	80,5%
RF Otimizado	82,5% 🏆

O Que Descobrimos

1

Quem já deu calote provavelmente vai dar de novo

Essa é a informação mais importante para a decisão (33% da importância total). Faz sentido: comportamento passado é o melhor indicador de comportamento futuro.

2

Só 5 informações importam de verdade

Das 22 variáveis, apenas 5 respondem por 88% da decisão. Gênero e escolaridade quase não influenciam - o que importa é comportamento financeiro.

3

O "melhor" modelo depende da estratégia do banco

RF Otimizado acerta mais, mas Random Forest perde menos dinheiro com calotes. Não existe resposta única - depende se o banco quer crescer ou ser conservador.

Recomendação Final

A escolha depende da estratégia do banco:

🚀 Banco quer crescer

Escolha: **RF Otimizado**
Aprova mais clientes bons, aceita um pouco mais de risco
F1: 82,5%

ou

🛡️ Banco quer segurança

Escolha: **Random Forest**
Perde menos com calotes, rejeita alguns bons clientes
F1: 80,5%

✅ **Nossa recomendação: RF Otimizado**

A diferença de impacto financeiro entre os dois é de apenas **~R$100 mil**. Por esse custo, o RF Otimizado identifica corretamente **2% mais clientes**, o que representa milhares de decisões melhores por ano.

92,8%	82,5%	97,5%
Acurácia	F1-Score	AUC

R$3M
Economia vs baseline